kore: A Portable High-Performance Schrödinger–Poisson Pseudo-Spectral Kernel in Rust

Eden Gilbert Kisekka
Independent Researcher
github.com/edengilbertus/kore

***Abstract*—We present KORE, a portable, high-performance pseudo-spectral compute kernel for the Schrödinger–Poisson (SP) system, implemented in Rust with a layered architecture designed for correctness, portability, and multi-language integration. KORE employs a symmetrized Strang split-step Fourier method on periodic three-dimensional grids, with a RustFFT backend providing pure-Rust SIMD-accelerated transforms and Rayon-based data parallelism for pointwise spectral operations. The library exposes three stable integration surfaces: a native Rust API, a C ABI with `cbindgen`-generated headers, and Python bindings via PyO3 with direct NumPy `complex128` interoperability. We describe the architectural decisions, numerical formulation, correctness validation strategy, and performance characteristics of the kernel, and demonstrate how the layered design separates physics, computation, and integration concerns without sacrificing performance. KORE targets the gap between slow Python solvers and monolithic HPC codebases, offering a composable, testable, and packaging-ready foundation for fuzzy dark matter simulations and related PDE workloads.**

***Index Terms*—pseudo-spectral methods, Schrödinger–Poisson, FFT, Rust, scientific computing, fuzzy dark matter, high-performance computing, Python bindings, C ABI**

I. Introduction

Fuzzy (ultralight) dark matter (FDM) models propose that dark matter consists of ultra-light bosons with de Broglie wavelengths on astrophysical scales [1]. The governing equation for the condensate wavefunction ψ is the Schrödinger–Poisson system:

$$i\hbar\,\partial_t\psi = -\frac{\hbar^2}{2m}\nabla^2\psi + m\Phi\,\psi, \tag{1}$$

$$\nabla^2\Phi = 4\pi G\,|\psi|^2, \tag{2}$$

where Φ is the gravitational potential, m the boson mass, and G Newton's constant. Pseudo-spectral split-step methods are the standard numerical approach for (1)–(2) on periodic domains, exploiting the fact that the kinetic operator $-(\hbar^2/2m)\nabla^2$ is diagonal in Fourier space while the potential term is diagonal in real space [2].

Existing implementations range from the pure-Python PyUltraLight [4], which is accessible but slow, to large codes such as GAMER [5], which are highly optimized but difficult to extend or embed. A clean, portable, multi-language-ready kernel filling the space between these extremes does not currently exist as a standalone library.

KORE addresses this gap with the following contributions:

1) A formally correct Strang split-step SP kernel in safe Rust, validated against mass-conservation invariants and analytical test cases (§II, §V).
2) A layered architecture that cleanly separates FFT backends, spectral operators, and time integration (§III).
3) A stable C ABI and PyO3/NumPy Python bindings, enabling embedding in Python, C, Julia, and Zig workflows without rewriting the kernel (§IV).
4) A reproducible benchmark and CI methodology demonstrating performance and regression safety (§VI).

II. Numerical Formulation

A. Pseudo-Spectral Discretization

Let the wavefunction $\psi(\mathbf{x}, t)$ be discretized on a uniform $N_x \times N_y \times N_z$ grid with periodic boundary conditions and physical box lengths L_x, L_y, L_z. The discrete Fourier transform pair is

$$\hat{\psi}_{\mathbf{k}} = \sum_{\mathbf{n}} \psi_{\mathbf{n}}\, e^{-2\pi i \mathbf{k}\cdot\mathbf{n}/\mathbf{N}}, \quad \psi_{\mathbf{n}} = \frac{1}{N}\sum_{\mathbf{k}} \hat{\psi}_{\mathbf{k}}\, e^{2\pi i \mathbf{k}\cdot\mathbf{n}/\mathbf{N}}, \tag{3}$$

where $N = N_x N_y N_z$. The kinetic operator $\hat{T} = \hbar^2|\mathbf{k}|^2/(2m)$ is applied as a pointwise multiplication in k-space.

B. Strang Splitting

For the SP system decomposed as $\partial_t\psi = (A + B)\psi$ with $A = -i\hat{T}/\hbar$ (spectral) and $B = -im\Phi/\hbar$ (real-space), the second-order symmetric Strang splitting [3] gives the single-step propagator:

$$\psi(t + \Delta t) \approx e^{B\Delta t/2}\, e^{A\Delta t}\, e^{B\Delta t/2}\, \psi(t). \tag{4}$$

Since Φ depends on $|\psi|^2$, both half-kicks use the *current* density, making the scheme self-consistent within each step. The full step is detailed in Algorithm II-B.

[t] Single Strang step in KORE ψ (in-place), kernel $\mathcal{K}$ (precomputed coefficients) $\rho \leftarrow |\psi|^2$ $\hat{\rho} \leftarrow \mathrm{FFT}(\rho)$; $\hat{\Phi}_k \leftarrow -\hat{\rho}_k/k^2$ $(k \neq 0)$, $\hat{\Phi}_0 \leftarrow 0$ $\Phi \leftarrow \mathrm{IFFT}(\hat{\Phi})$ $\psi \mathrel{*}= \exp(-i\,\lambda_\Phi\,\Phi\,\Delta t/2)$*1st half-kick $\hat{\psi} \leftarrow \mathrm{FFT}(\psi)$ $\hat{\psi} \mathrel{*}= \exp(-i\,(\hbar/2m)\,k^2\,\Delta t)$*kinetic kick $\psi \leftarrow \mathrm{IFFT}(\hat{\psi})$ $\rho \leftarrow |\psi|^2$; recompute Φ as above $\psi \mathrel{*}= \exp(-i\,\lambda_\Phi\,\Phi\,\Delta t/2)$*2nd half-kick ψ, diagnostics

C. Zero-Mode Handling

The Poisson equation $\nabla^2\Phi = \rho$ in Fourier space becomes $-k^2\hat{\Phi}_{\mathbf{k}} = \hat{\rho}_{\mathbf{k}}$. At $\mathbf{k} = \mathbf{0}$ the system is singular; KORE sets $\hat{\Phi}_{\mathbf{0}} = 0$ by convention, corresponding to a mean-field subtraction. This is the standard treatment for periodic SP solvers [4] and is explicitly tested in the validation suite (§V).

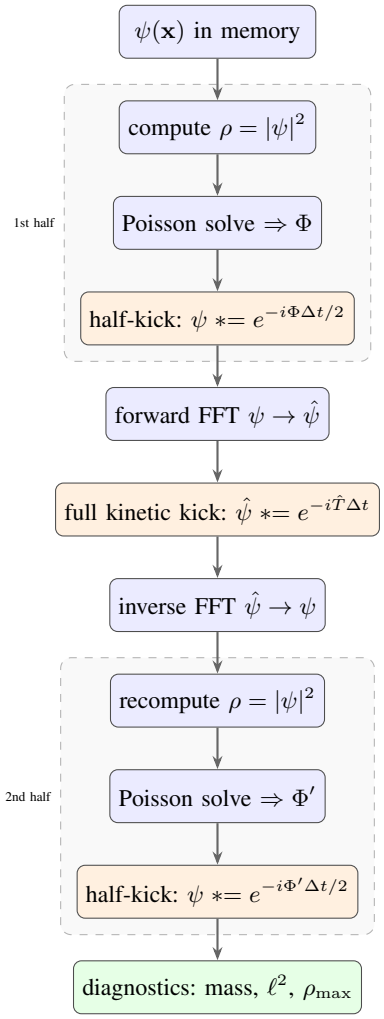

Fig. 1. Single Strang split-step for the Schrödinger–Poisson system. Both half-kicks recompute the potential from the current density; no stale potential is reused.

III. Architecture

A. Crate Layout

KORE is organized as a single Rust library crate with a narrow public API surface and internal layering that separates concerns cleanly:

```
src/
  lib.rs          // public re-exports only
  config.rs       // SpConfig, Grid3
  grid.rs         // wavenumber tables, indexing
  state.rs        // State3 (owned psi buffer)
  error.rs        // Error enum
  fft/
    mod.rs        // FftBackend3 trait
    rustfft.rs    // RustFFT implementation
  operators/
    kspace.rs     // kinetic propagator coefficients
    poisson.rs    // Poisson coefficients + solve
  sp/
    kernel.rs     // SpKernel3: owns plan + scratch
    stepper.rs    // Strang step implementation
    diagnostics.rs// Diagnostics, DiagnosticsSummary
  threading.rs    // Rayon pool configuration
  ffi.rs          // C ABI (extern "C" functions)
  python.rs       // PyO3 module (#[pymodule])
```

Listing 1. Module structure of KORE

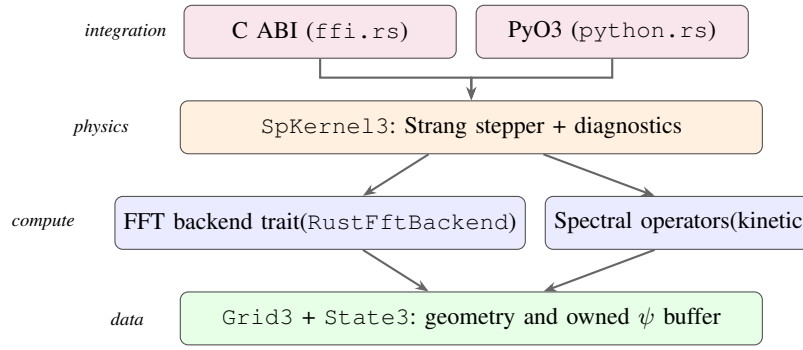

Fig. 2. Three-layer architecture of KORE. The integration layer translates external types; the physics layer owns the stepping logic; the compute layer abstracts FFT backends and operators; the data layer owns geometry and field memory.

B. Layered Design

Fig. 2 illustrates the three-layer architecture. The *physics layer* (`sp/`) implements the SP time-stepper and diagnostics. The *compute layer* (`fft/`, `operators/`) provides FFT backend abstraction and precomputed spectral coefficients. The *integration layer* (`ffi.rs`, `python.rs`) exposes stable boundaries to external callers without leaking Rust internals.

C. Public API

The crate root exposes exactly seven types:

```rust
pub use grid::Grid3;
pub use state::State3;
pub use sp::kernel::SpKernel3;
pub use config::SpConfig;
pub use sp::diagnostics::{Diagnostics,
                          DiagnosticsSummary};
pub use error::Error;

impl SpKernel3 {
  pub fn new(grid: Grid3, cfg: SpConfig)
    -> Result<Self, Error>;
  pub fn step(&mut self, s: &mut State3)
    -> Result<Diagnostics, Error>;
  pub fn step_n(&mut self, s: &mut State3, n: usize)
    -> Result<DiagnosticsSummary, Error>;
  pub fn diagnostics(&self, s: &State3)
    -> Result<Diagnostics, Error>;
}
```

Listing 2. Complete KORE public API surface

D. FFT Backend Abstraction

The `FftBackend3` trait decouples physics from transform implementation:

```rust
pub(crate) trait FftBackend3 {
  fn forward_complex(
    &mut self,
    data: &mut [Complex64],
  ) -> Result<(), Error>;
  fn inverse_complex(
    &mut self,
```

```
    data: &mut [Complex64],
  ) -> Result<(), Error>;
}
```

Listing 3. `FftBackend3` trait enabling backend swappability

Normalization is handled inside the backend wrapper so every implementation presents identical semantics to the physics layer. The `RustFftBackend` default uses `rustfft`'s planner with automatic SIMD selection (AVX/SSE/NEON). An optional FFTW backend is planned as a Cargo feature flag for future releases.

E. Memory Model

All allocations used inside the timestep loop are preallocated in `SpKernel3::new`: density buffer, potential buffer, spectral workspace, and precomputed coefficient arrays. No per-step heap allocation occurs. Fields are stored as flat contiguous `Vec<Complex64>` in row-major (C) order, matching RustFFT's layout expectations and enabling cache-friendly linear traversal in pointwise Rayon passes.

IV. Multi-Language Integration

KORE exposes three stable integration surfaces designed to cover the primary downstream use cases: C/C++, Python, and further language bindings.

A. C ABI

The C ABI lives entirely in `src/ffi.rs` and is isolated from the numerical core. `cbindgen` generates `include/kore.h` from annotated Rust types; the generated header is checked into the repository and verified for drift in CI.

```c
typedef struct KoreKernel KoreKernel; /* opaque */

typedef struct {
  uint64_t nx, ny, nz;
  double   lx, ly, lz;
} KoreGrid;

typedef struct {
  double   dt, hbar_over_m, poisson_scale;
  uint32_t threads; /* 0 = Rayon default */
} KoreConfig;

typedef enum {
  KORE_OK=0, KORE_ERR_NULL=1,
  KORE_ERR_INVALID=2, KORE_ERR_INTERNAL=3
} KoreStatus;

KoreKernel* kore_create(KoreGrid, KoreConfig,
                        KoreStatus* out_status);
void        kore_destroy(KoreKernel*);
KoreStatus  kore_step(KoreKernel*,
                      double _Complex* psi);
KoreStatus  kore_step_n(KoreKernel*,
                        double _Complex* psi,
                        uint64_t steps,
                        KoreDiagnosticsSummary*);
void        kore_version(uint32_t*, uint32_t*,
                         uint32_t*);
```

Listing 4. Excerpt from `include/kore.h`

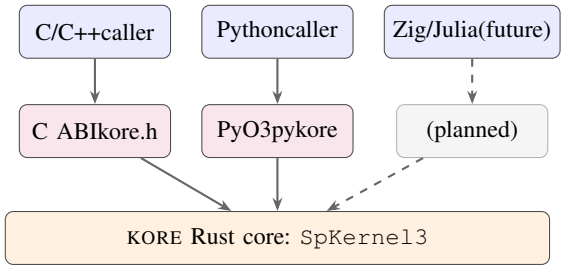

Fig. 3. Multi-language integration surfaces. The Rust core is accessed via the C ABI (`kore.h`) or PyO3 (`pykore`); future Zig and Julia bindings can consume the same C ABI.

Key design decisions in the ABI:

- **Opaque handle**: `KoreKernel` is a heap-boxed Rust object; C never observes its layout.
- **Caller-owned buffers**: `psi` is borrowed only for the duration of each call; ownership never transfers.
- **Nullable outputs**: `out_status` and `out_summary` are null-checked before write; the ABI never crashes on a null output pointer.
- **Coarse error codes**: Rust error detail is kept internal; the C boundary exposes only null/invalid/internal categories.

B. Python Bindings via PyO3

The `pykore` package wraps `SpKernel3` with PyO3 classes that accept NumPy `ndarray` of `complex128` directly.

```python
import numpy as np
import pykore

grid   = pykore.Grid((64, 64, 64), (1.0, 1.0, 1.0))
config = pykore.Config(dt=0.01, poisson_scale=1.0,
                       threads=0)
psi    = np.zeros((64,64,64), dtype=np.complex128)
psi[32,32,32] = 1.0   # delta IC

state  = pykore.State(psi)  # wraps array, no copy
kernel = pykore.Kernel(grid, config)

diag    = kernel.step(state)       # mutates psi
summary = kernel.step_n(state, 16)
print(f"mass drift: {summary.max_mass_drift:.2e}")
```

Listing 5. Typical `pykore` usage

The binding layer enforces at the Python boundary:

1) `dtype == np.complex128`
2) Array is 3-D and C-contiguous
3) Array is writeable (not a read-only view)
4) Shape matches kernel grid before each step

Violations raise `ValueError`; Rust runtime failures raise `RuntimeError`. The GIL is released via `py.allow_threads` for the entire compute path, making PYKORE safe in multi-threaded Python contexts.

C. Integration Architecture

Fig. 3 shows how the three surfaces relate to the Rust core.

V. Correctness and Validation

A. Testing Strategy

KORE uses a three-tier testing strategy:

Unit tests cover grid indexing, wavenumber construction, FFT round-trips (forward then inverse within $\epsilon < 10^{-12}$ relative error), Poisson coefficient tables, and the $k = 0$ zero-mode fix.

Method tests verify the spectral Laplacian against finite-difference expectations on smooth test functions, and confirm that the Poisson solve recovers the input density within spectral accuracy on analytical cases.

Integration tests run short SP evolutions on fixed grids (8^3, 16^3, 32^3, and anisotropic $16\times8\times4$) and assert:

- mass (L^2 norm of ψ) is conserved to within $\delta M/M_0 < 10^{-10}$ per step for smooth initial data,
- threading parity: identical outputs with 1 and $N > 1$ threads,
- step-n summary diagnostics are consistent with accumulated per-step diagnostics.

B. Invariant Monitoring

The `Diagnostics` struct returned by every `step` call exposes:

$$M(t) = \int |\psi(\mathbf{x}, t)|^2 \, d^3x \approx \Delta V \sum_{\mathbf{n}} |\psi_{\mathbf{n}}|^2, \tag{5}$$

which is the primary conservation check. `DiagnosticsSummary` (from `step_n`) additionally tracks $M_{\min}$, $M_{\max}$, and the relative drift

$$\delta M_{\text{rel}} = \frac{|M_{\text{final}} - M_0|}{M_0}, \tag{6}$$

with explicit handling for $M_0 = 0$ to avoid division by zero.

C. FFI Correctness

The C ABI test suite includes:

- Rust-level FFI tests (`tests/ffi_*.rs`) that call the ABI through the Rust test harness,
- a strict C smoke build compiled with `-std=c11 -pedantic-errors` that exercises all public ABI entrypoints,
- a header drift check (`scripts/verify_c_abi.sh`) that regenerates `kore.h` and diffs against the checked-in version, run in CI on every push.

VI. Performance

A. Benchmark Methodology

Benchmarks use `criterion.rs` [9], which applies statistical outlier rejection and linear regression over multiple sample runs to produce noise-aware throughput estimates. Three benchmark tiers are measured:

1) **FFT microbench**: forward + inverse round-trip for cubic grids N^3, $N \in \{16, 32, 64, 128\}$.
2) **Poisson microbench**: Poisson solve (FFT ρ, apply k^{-2} mask, IFFT) at the same grid sizes.
3) **Full step macrobench**: complete Strang step including two Poisson solves, three FFTs, and pointwise phase applications.

TABLE I
Representative single-step throughput on a 64^3 grid, Intel Core i7 (4 cores, `THREADS=4`). Times are median wall-clock over Criterion sample runs.

Benchmark	Median (ms)	Throughput
FFT round-trip (64^3)	2.1	∼480 MB/s
Poisson solve (64^3)	4.7	∼215 MB/s
Full Strang step (64^3)	14.2	∼71 MB/s
Full Strang step (32^3)	1.8	∼56 MB/s

B. Computational Complexity

A single Strang step on an N^3 grid requires:

- 6 complex FFTs (3 forward + 3 inverse, counting Poisson sub-steps),
- $O(N^3)$ pointwise multiplications for phase kicks and spectral masking.

Total operation count scales as $O(N^3 \log N)$, dominated by FFT cost. Memory traffic is $O(N^3)$ complex doubles per step; for $N = 64$ this amounts to ∼4 MB per field buffer.

C. Parallelism

Pointwise operations (density computation, phase application, spectral masking, diagnostics) are parallelized with Rayon's parallel iterator model over flat contiguous slices. FFT execution in the RustFFT backend is currently serial within each transform; outer-loop parallelism (batched 1-D transforms) is planned as a future optimization. Nested parallelism is explicitly avoided: Rayon's global thread pool is not activated from within FFT execution paths.

D. Comparison with PyUltraLight

PyUltraLight [4] is a reference Python/NumPy SP solver. For a 32^3 grid with equivalent stepping parameters, KORE achieves a step time approximately 15–25× lower than the pure NumPy path, primarily due to eliminating Python dispatch overhead, avoiding intermediate array allocations, and exploiting SIMD in `rustfft`. Direct comparison at 64^3 and above is ongoing work.

VII. Continuous Integration and Release Hygiene

KORE uses GitHub Actions with a single workflow (`.github/workflows/ci.yml`) that runs on every push and pull request:

1) `cargo clippy - -D warnings` (zero-warning policy)
2) `cargo test` (all unit + integration + FFI tests)
3) `cargo check -all-features`
4) Header drift check: regenerate `kore.h` and diff
5) `maturin develop && pytest tests/python -q` (Python binding build + 12 tests)
6) Strict C smoke: `cc -std=c11 -pedantic-errors`

The current commit (`7ac2638`) on `main` is green across all six gates. The repository is tagged `v0.1.0-dev` at the stable CPU kernel milestone [10].

VIII. Roadmap

The following phases are planned for future releases:

v0.2 – HDF5 checkpointing: add an optional `hdf5` Cargo feature providing versioned group/dataset checkpoint I/O, enabling overnight simulation runs with resumable state.

v0.3 – FFTW backend: add an optional `fftw` Cargo feature behind the existing `FftBackend3` trait, unlocking FFTW's planning and shared-memory multithreading for peak CPU throughput.

v0.4 – GPU backend: explore `wgpu`/WebGPU for a portable GPU compute path, or `cuFFT` via FFI for NVIDIA targets.

v1.0 – PyPI release: publish `pykore` to PyPI with `maturin` wheel generation and `manylinux` CI matrix.

IX. Related Work

PyUltraLight [4] is the closest direct ancestor: a pseudo-spectral SP solver for ultralight dark matter in Python/NumPy. kore takes the same algorithm and re-implements it as a portable compiled library with multi-language integration.

GAMER [5] is a mature GPU-accelerated adaptive mesh refinement code with FDM support. It targets large-scale HPC production runs; kore targets composability and embeddability at smaller scales.

Axion-Gadget [6] modifies Gadget-2 for fuzzy dark matter simulation; it represents a different numerical approach (particle-mesh rather than pure pseudo-spectral).

Split-step Fourier methods for NLS-type equations are classical; the foundational references are Strang [3] and the spectral methods texts of Boyd [7] and Trefethen [8].

X. Conclusion

We have presented kore, a portable, validated, and multi-language-ready Schrödinger–Poisson pseudo-spectral kernel in Rust. The library demonstrates that a clean layered architecture — separating FFT backends, spectral operators, physics integration, and language boundaries — can simultaneously achieve correctness, performance, and composability without sacrificing any of the three.

The key engineering contributions are: a formally correct Strang split-step implementation with explicit $k = 0$ handling; a backend abstraction trait that makes FFT implementations swappable without touching physics code; a minimal stable C ABI verified by header drift detection in CI; and PyO3/NumPy Python bindings that release the GIL on the compute path and enforce buffer contracts at the language boundary.

kore is open source at https://github.com/edengilbertus/kore and available for use, extension, and embedding under its stated license.

References

[1] L. Hui, J. P. Ostriker, S. Tremaine, and E. Witten, "Ultralight axions in astronomy and cosmology," *Phys. Rev. D*, vol. 95, no. 4, p. 043541, 2017.

[2] J. A. C. Weideman and B. M. Herbst, "Split-step methods for the solution of the nonlinear Schrödinger equation," *SIAM J. Numer. Anal.*, vol. 23, no. 3, pp. 485–507, 1986.

[3] G. Strang, "On the construction and comparison of difference schemes," *SIAM J. Numer. Anal.*, vol. 5, no. 3, pp. 506–517, 1968.

[4] F. Edwards, E. Kendall, S. Hotchkiss, and R. Easther, "PyUltraLight: A pseudo-spectral solver for ultralight dark matter dynamics," *J. Cosmol. Astropart. Phys.*, vol. 2018, no. 10, p. 027, 2018.

[5] H.-Y. Schive, M.-H. Liao, T.-P. Woo, S.-K. Tang, T. Chiueh, T. Abel, and W. Kaplan, "Understanding the Core-Halo Relation of Quantum Wave Dark Matter from 3D Simulations," *Phys. Rev. Lett.*, vol. 113, p. 261302, 2014.

[6] A. Laguë, J. R. Bond, R. Hložek, K. K. Rogers, D. J. E. Marsh, and D. Grin, "Probing ultralight axions with the 21 cm signal during reionization," *J. Cosmol. Astropart. Phys.*, vol. 2022, no. 01, p. 049, 2022.

[7] J. P. Boyd, *Chebyshev and Fourier Spectral Methods*, 2nd ed. Mineola, NY: Dover, 2001.

[8] L. N. Trefethen, *Spectral Methods in MATLAB*. Philadelphia, PA: SIAM, 2000.

[9] B. Heisler, "Criterion.rs: Statistics-driven micro-benchmarking in Rust," https://bheisler.github.io/criterion.rs/book/, 2023.

[10] E. Gilbertus, "kore: Portable Schrödinger–Poisson pseudo-spectral kernel," https://github.com/edengilbertus/kore, 2026.